

MMN

Modular Reconnaissance & Assessment Framework

Comprehensive Technical Report

Architecture · Features · Methodology · Module Reference

Version	2.0.0
Report Date	September 28, 2026
Author	Md. Mostofa Nayon
Affiliation	Professional Master's in Information & Cyber Security, University of Dhaka
Contact	contact@nayon.bd · +8801301994411
Web / Code	nayon.bd · github.com/MMNayon00
Classification	Educational / Authorized Security Assessment

Table of Contents

Table of Contents	2
1. Executive Summary	4
1.1 Key Capabilities at a Glance	4
2. Introduction & Ethical Framework	5
2.1 Purpose & Scope	5
2.2 Authorization Model	5
2.3 What MMN Does Not Do	5
2.4 Positioning in the Assessment Lifecycle	5
3. System Architecture	6
3.1 Design Principles	6
3.2 Repository Layout	6
4. Installation & Execution	8
4.1 Requirements	8
4.2 Step-by-Step Setup (Kali / Linux / macOS)	8
4.3 Kali (PEP 668) Note	8
4.4 Dependencies	8
5. Operating Methodology	9
5.1 Reconnaissance Phases	9
5.2 Data Flow	9
6. Module Reference — How Each Feature Works	10
6.1 Input Handler	10
6.2 Target Expansion	10
6.3 Footprinting	11
6.4 Subdomain Enumeration	11
6.5 Port & Service Enumeration	11
6.6 Technology Fingerprinting	12
6.7 CVE Vulnerability Mapping	12
6.8 JS Hidden Document Intelligence (v2)	13
6.9 Report Generator	13

7. Risk Scoring & Severity Model

7.1 CVSS Severity Bands

7.2 JS Intelligence Risk & Confidence

8. Reporting Engine

9. Limitations & Future Work

9.1 Known Limitations

9.2 Planned Enhancements

9.3 Verification Status

10. About the Author

Appendix A — Common Ports Reference

Appendix B — Glossary

15

15

15

16

17

17

17

18

19

19

1. Executive Summary

MMN (Modular Reconnaissance & Assessment Framework) is a Python-based, passive-leaning reconnaissance and vulnerability-assessment platform built for **authorized** penetration testing and security education. It automates the early phases of an engagement — target expansion, footprinting, asset discovery, service enumeration, technology fingerprinting, CVE correlation, and JavaScript-based hidden-document intelligence — and consolidates all findings into four report formats (CLI, JSON, HTML, and PDF).

The framework performs **identification and assessment only**. It does not exploit, brute-force credentials, or launch denial-of-service. Every run begins with an explicit authorization gate. This document describes the complete architecture, the operating methodology, and each of the nine modules in detail, supported by diagrams, charts, and reference tables.

1.1 Key Capabilities at a Glance

Capability	Delivered By	Primary Output
DNS & IP expansion	target_expansion	A/AAAA/MX/NS/TXT/SOA records, IPs, reverse DNS
WHOIS / TLS / HTTP footprint	footprinting	Registrar, certificate, headers, security posture
Subdomain discovery	subdomain_enum	Unique subdomains from 3 passive sources
Port & service enumeration	port_service_enum	Open ports, banners, versions, OS guess
Technology fingerprinting	tech_fingerprint	Servers, CMS, frameworks, JS libraries
Vulnerability correlation	cve_mapper	CVEs with CVSS scores & severity
JS hidden-document intel (v2)	js_hidden_doc_intel	Endpoints, exposed docs, PDF metadata
Multi-format reporting	report_generator	CLI, JSON, HTML dashboard, PDF

Table 1.1 — Capability-to-module mapping.

2. Introduction & Ethical Framework

2.1 Purpose & Scope

MMN targets the *reconnaissance* and *enumeration* phases of the penetration-testing lifecycle. It is intended for systems the operator owns or is contractually authorized to test, for controlled lab environments, and for in-scope bug-bounty programs.

2.2 Authorization Model

Execution is gated: the operator must explicitly confirm authorization before a target is accepted, and a legal disclaimer is displayed at startup. Scope enforcement is also applied inside the JS intelligence module, which only records endpoints and documents belonging to the authorized domain set.

2.3 What MMN Does Not Do

- No exploitation or payload delivery.
- No credential brute-forcing.
- No denial-of-service or traffic flooding.
- No modification of remote systems — read-only banner and metadata collection.
- No authentication bypass or fuzzing in the JS intelligence module (strict passive mode).

2.4 Positioning in the Assessment Lifecycle



Feedback loop: operator may re-scan with different modules on the same target

Figure 2.1 — MMN operating workflow, from authorization to reporting.

3. System Architecture

MMN follows a modular controller-and-plugins design. A thin controller (`main.py`) handles the banner, authorization gate, menu, and orchestration, then delegates work to independent, single-responsibility modules under `modules/`. Each module returns a normalized Python dictionary; the controller aggregates these into a single `results` structure that the reporting engine renders.

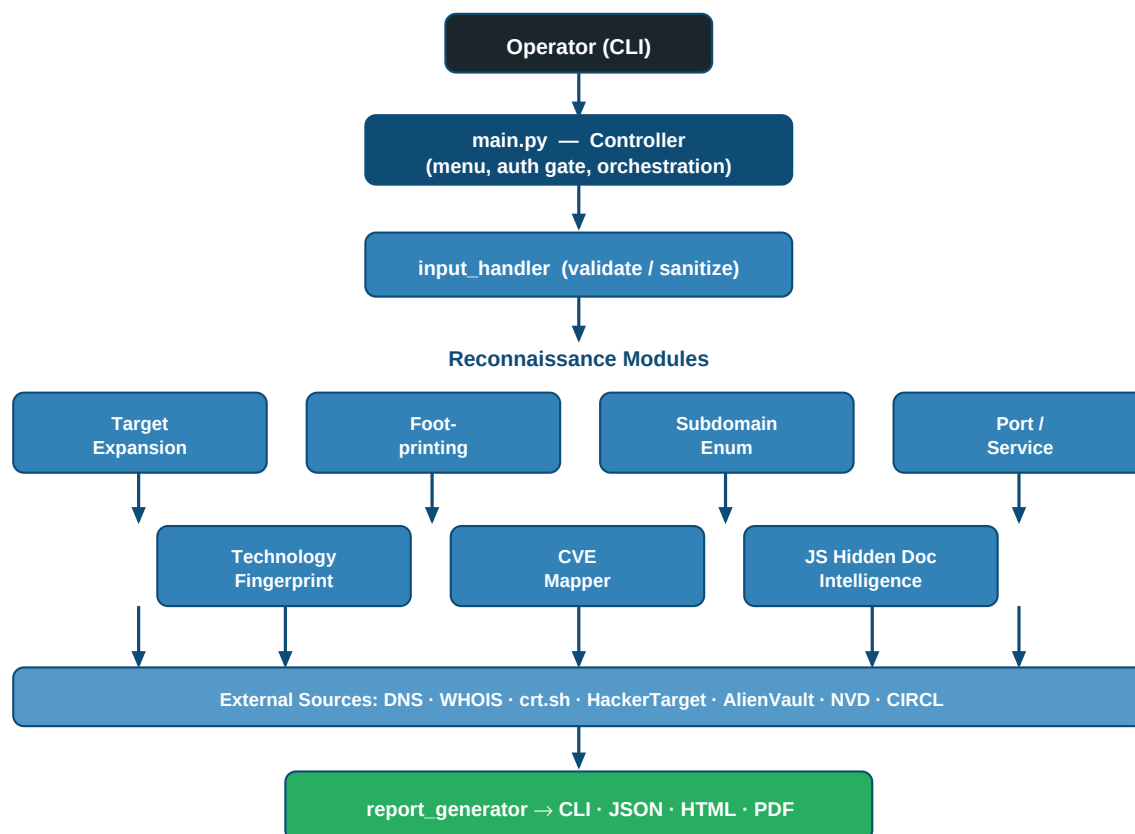


Figure 3.1 — High-level architecture and external data sources.

3.1 Design Principles

- **Separation of concerns:** each module owns one reconnaissance domain and can run standalone.
- **Graceful degradation:** a failed source or timeout never aborts the whole scan; modules return partial, well-formed data.
- **Normalized data contract:** every module emits predictable dictionary keys, decoupling collection from reporting.
- **Rate limiting & concurrency:** thread pools accelerate scanning while configurable delays reduce target load.
- **Defensive parsing:** all network input is treated as untrusted; report output is HTML-escaped.

3.2 Repository Layout

Path	Responsibility
<code>main.py</code>	Controller: banner, auth gate, menu, orchestration
<code>modules/input_handler.py</code>	Target validation, sanitization, resolution
<code>modules/target_expansion.py</code>	DNS records, IP discovery, reverse DNS, hosting

Path	Responsibility
modules/footprinting.py	WHOIS, TLS certificate, HTTP & security headers
modules/subdomain_enum.py	crt.sh, HackerTarget, AlienVault, optional brute force
modules/port_service_enum.py	Port scan, banner grab, version & OS detection
modules/tech_fingerprint.py	Web server / CMS / framework / library detection
modules/cve_mapper.py	NVD + CIRCL CVE lookup, CVSS scoring
modules/js_hidden_doc_intel.py	JS endpoint/document discovery, PDF metadata (v2)
modules/report_generator.py	CLI, JSON, HTML, PDF report generation
reports/	Generated reports & audit logs (git-ignored)

Table 3.1 — Project structure.

4. Installation & Execution

4.1 Requirements

- Python 3.8 or newer.
- pip package manager.
- Internet access for DNS, WHOIS, Certificate Transparency, and CVE lookups.

4.2 Step-by-Step Setup (Kali / Linux / macOS)

```
git clone https://github.com/MMNayon00/Kali-Tools.git
cd Kali-Tools
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
python3 main.py
```

4.3 Kali (PEP 668) Note

Recent Kali/Debian builds block system-wide pip installs. The virtual-environment steps above avoid this. To install system-wide instead, append `--break-system-packages` to the install command.

4.4 Dependencies

Package	Version	Used For
colorama	0.4.6	Colored CLI output
requests	2.31.0	HTTP client for all web lookups
dnspython	2.4.2	DNS record resolution
python-whois	0.8.0	WHOIS registrar data
beautifulsoup4	4.12.2	HTML parsing / tech fingerprinting
reportlab	4.0.7	PDF report generation
pypdf	≥ 3.17.0	PDF metadata extraction (JS intel)

Table 4.1 — Third-party dependencies.

5. Operating Methodology

MMN offers three run modes from the main menu: **Full Assessment** (all modules in sequence), **Basic Footprinting** (expansion + footprinting for a quick profile), and **Custom Selection** (any subset of modules). The recommended methodology mirrors a real engagement, moving from broad, low-noise passive collection toward more specific active enumeration.

5.1 Reconnaissance Phases

Phase	Modules	Nature	Goal
1. Expansion	target_expansion	Passive	Map DNS surface & IP footprint
2. Footprinting	footprinting	Passive	Registrar, TLS, HTTP security posture
3. Asset discovery	subdomain_enum	Passive	Enumerate attack surface
4. Enumeration	port_service_enum	Active	Open ports, service versions, OS
5. Fingerprinting	tech_fingerprint	Active	Identify web technology stack
6. Correlation	cve_mapper	Passive*	Map versions to known CVEs
7. Deep intel	js_hidden_doc_intel	Passive	Hidden endpoints & leaked documents

Table 5.1 — Phased methodology. *CVE lookups query public APIs, not the target.

5.2 Data Flow

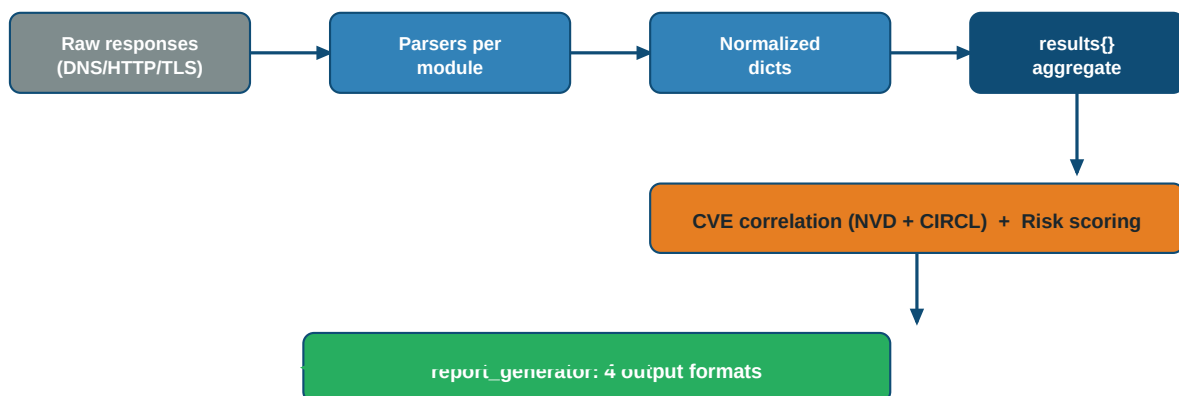


Figure 5.1 — From raw responses to correlated, scored, multi-format reports.

6. Module Reference — How Each Feature Works

6.1 Input Handler

modules/input_handler.py

Purpose

Validates and sanitizes the operator-supplied target and resolves domains to IP addresses so downstream modules receive clean, trusted input.

How It Works

Regex validation distinguishes IPv4 addresses (with per-octet 0–255 range checks) from domain names. Input is stripped of protocol prefixes, trailing slashes, and shell metacharacters before use. Domain-to-IP resolution uses the system resolver with a DNS fallback.

Methodology

- IPv4 and domain format validation with range checking.
- Sanitization removes ; & | ` \$ () to neutralize injection.
- Interactive retry loop on invalid input.
- Resolution returns the target unchanged if it is already an IP.

Data Sources	Key Output Fields
Operator input System DNS resolver	validated target (str) resolved IPv4 address

6.2 Target Expansion

modules/target_expansion.py

Purpose

Builds the DNS and IP picture of the target: record types, all resolved IPs, reverse DNS, and a best-effort hosting-provider label.

How It Works

Queries Google and Cloudflare resolvers for A, AAAA, MX, NS, TXT, CNAME, and SOA records. All A-record IPs are collected via getaddrinfo with a dnspython fallback. The primary IP is reverse-resolved, and its PTR hostname is matched against a table of known cloud providers.

Methodology

- Multi-resolver DNS queries (8.8.8.8, 1.1.1.1) with timeouts.
- Complete record-type sweep (A/AAAA/MX/NS/TXT/CNAME/SOA).
- Deduplicated multi-IP discovery.
- Provider inference from reverse-DNS keywords (AWS, GCP, Cloudflare, Azure, ...).

Data Sources	Key Output Fields
Google/Cloudflare DNS System resolver Reverse DNS (PTR)	dns_records{} ip_addresses[] reverse_dns hosting_provider{}

6.3 Footprinting

modules/footprinting.py

Purpose

Profiles the target's ownership, encryption, and HTTP security posture without touching application logic.

How It Works

Performs a WHOIS lookup (registrar, dates, name servers, contacts), inspects the TLS certificate via a verifying handshake so subject/issuer/validity/SAN fields populate correctly, and analyzes HTTP response headers — grading the presence of five key security headers.

Methodology

- WHOIS parsing that tolerates string/list/datetime field variants.
- TLS inspection with verify-first, unverified-fallback so certificate data is never silently empty.
- HTTPS-first with automatic HTTP fallback.
- Security-header scorecard (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection).

Data Sources	Key Output Fields
WHOIS registries TLS handshake (port 443) HTTP(S) headers	whois{} ssl_certificate{} http_headers{} + security_headers{}

6.4 Subdomain Enumeration

modules/subdomain_enum.py

Purpose

Discovers subdomains — and therefore additional attack surface — using passive, non-intrusive public data sources.

How It Works

Concurrently queries three sources: crt.sh Certificate Transparency logs, the HackerTarget host-search API, and AlienVault OTX passive DNS. Results are validated against the root domain, wildcard/whitespace entries are rejected, and the union is deduplicated. An optional wordlist brute force is available but off by default.

Methodology

- Three independent passive sources for breadth and resilience.
- Retry-with-backoff HTTP helper for flaky endpoints.
- Strict membership check so only true sub/domains are kept.
- Rate-limit / error-message detection for API sources.

Data Sources	Key Output Fields
crt.sh (CT logs) HackerTarget API AlienVault OTX	subdomains[] (deduplicated, sorted)

6.5 Port & Service Enumeration

modules/port_service_enum.py

Purpose

Identifies open TCP ports, grabs service banners, infers product/version, and estimates the operating system.

How It Works

A thread pool connects to a common-port set (or a custom/full range) using non-blocking connect checks. Open ports are banner-grabbed — TLS-wrapped where needed — and matched against service signatures (SSH, HTTP servers, FTP, SMTP). OS detection reads the target's real ICMP TTL and rounds up to the nearest initial value (64/128/255) to account for hops.

Methodology

- Concurrent scanning with per-completion rate limiting and live ETA.
- TLS-aware banner grabbing (443/8443/465/993/995/636).
- Regex version extraction for Apache, nginx, IIS, OpenSSH, ...
- Hop-aware TTL fingerprinting plus port-based OS hints (RDP/SMB/AFP).

Data Sources	Key Output Fields
TCP connect probes Service banners ICMP echo (TTL)	ports[] (port/service/product/version/banner) os_detection{}

6.6 Technology Fingerprinting

modules/tech_fingerprint.py

Purpose

Determines the web technology stack — servers, CDNs, CMS platforms, frameworks, and JavaScript libraries.

How It Works

Fetches the target page and inspects both HTTP headers (Server, X-Powered-By, generator hints) and HTML body (meta generators, script/style signatures). Each finding carries a confidence level so the operator can weigh evidence strength.

Methodology

- Header-based detection (Apache, nginx, IIS, Cloudflare, PHP, ASP.NET).
- HTML/meta detection (WordPress, Drupal, Joomla).
- Library detection (jQuery, React, Vue, Angular, Bootstrap, Analytics).
- Confidence rating (High/Medium) per technology.

Data Sources	Key Output Fields
HTTP response headers Rendered HTML body	technologies[] (name/category/confidence)

6.7 CVE Vulnerability Mapping

modules/cve_mapper.py

Purpose

Correlates discovered products and versions with publicly known vulnerabilities and ranks them by severity.

How It Works

For each identified service it queries the NVD 2.0 API (primary) and falls back to CIRCL when results are sparse. CVEs are deduplicated, CVSS v3.1/v3.0/v2.0 scores are extracted, severity is normalized, and findings

are sorted by score. NVD rate limits are respected with built-in delays.

Methodology

- Dual-source lookup (NVD primary, CIRCL fallback) with de-duplication.
- CVSS v3.1 → v3.0 → v2.0 precedence for scoring.
- Severity normalization (NONE/UNKNOWN folded to Informational) — no counting crashes.
- Retry/backoff on HTTP 429/5xx; results sorted highest-CVSS first.

Data Sources	Key Output Fields
NVD 2.0 API CIRCL CVE Search	cve_report{} (totals, severity_counts, findings[])

6.8 JS Hidden Document Intelligence (v2)

modules/js_hidden_doc_intel.py

Purpose

The flagship v2 module: mines HTML and JavaScript for undocumented endpoints, exposed non-linked documents, outdated libraries, and leaked PDF metadata — in strict passive mode.

How It Works

It collects external and inline scripts, then applies regex intelligence to extract REST/GraphQL/WebSocket endpoints, hard-coded URLs, and document references. Document references are verified with HEAD requests (200-only), confirmed PDFs are parsed for metadata (author, producer, embedded URLs), and library versions are checked against known-current releases. Every finding is risk-classified, an audit log is written, and a confidence score (0–100) is computed.

Methodology

- Concurrent script collection with thread-safe de-duplication.
- Endpoint patterns: REST, GraphQL ops, WebSocket, Swagger/OpenAPI, document paths.
- Scope enforcement — only in-scope endpoints/documents are recorded.
- HEAD-only verification (no downloads except confirmed PDFs for metadata).
- Risk model: critical/high/medium/low/informational + confidence score.

Data Sources	Key Output Fields
Target HTML External & inline JS HEAD verification PDF metadata	endpoints_discovered[] hidden_documents_found[] pdf_metadata_extracted[] outdated_libraries[] risk_summary{} confidence_score

6.9 Report Generator

modules/report_generator.py

Purpose

Aggregates all module output into four report formats and guarantees safe, complete rendering.

How It Works

Produces a colored CLI summary, a machine-readable JSON file, a self-contained HTML dashboard (summary cards, navigation, and every module's findings), and a professional PDF. Filenames are sanitized, values are

HTML-escaped, and None/empty fields degrade to dashes rather than errors.

Methodology

- Four formats: CLI, JSON, HTML dashboard, PDF.
- Full HTML coverage of every module incl. JS intel & PDF metadata.
- Filesystem-safe filename slugs; XSS-safe HTML escaping.
- Robust to missing/None sections across all report types.

Data Sources	Key Output Fields
Aggregated results{}	*.json *.html *.pdf CLI output

7. Risk Scoring & Severity Model

MMN uses the industry-standard CVSS scale for vulnerability severity and a separate qualitative risk model for JS-intelligence findings. The two are presented side by side so an operator can triage quickly.

7.1 CVSS Severity Bands

Severity	CVSS Range	Operator Action
CRITICAL	9.0 – 10.0	Immediate patching required
HIGH	7.0 – 8.9	Prioritize remediation
MEDIUM	4.0 – 6.9	Schedule remediation
LOW	0.1 – 3.9	Track / accept with review
INFORMATIONAL	0.0 / N/A	Awareness only

Table 7.1 — CVSS v3.1 severity bands and recommended actions.

Illustrative CVSS v3.1 Severity Distribution (sample scan)

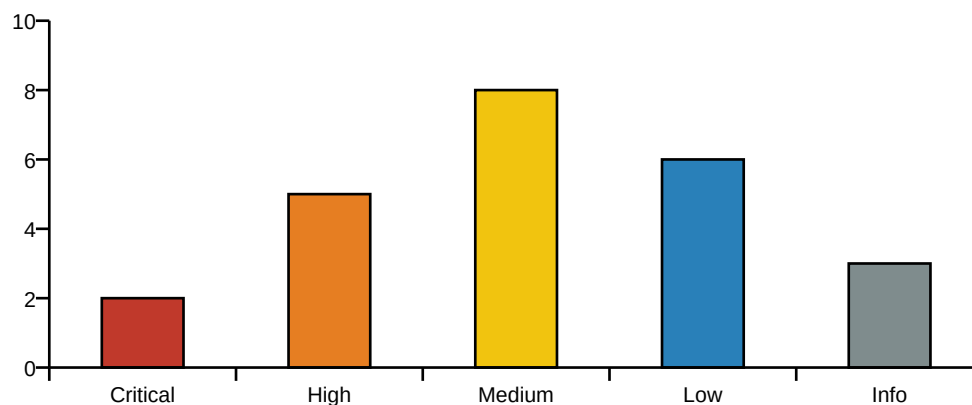


Figure 7.1 — Example severity distribution from a sample assessment.

7.2 JS Intelligence Risk & Confidence

Findings from the JS module are tagged critical/high/medium/low/informational based on the endpoint type and document sensitivity (for example, exposed .sql/.bak files or admin routes rank higher). A confidence score (0–100) reflects how much JS surface was successfully collected and analyzed, giving the operator a sense of coverage completeness.

8. Reporting Engine

The same normalized results power four outputs. The HTML dashboard is the richest: a summary card grid plus a navigable section for every module that ran, including full per-service CVE detail and the complete JS-intelligence section with PDF metadata.

Format	Audience	Contents	Full Assessment	Basic / Custom
CLI	Operator (live)	Colored on-screen summary	Yes	Yes
JSON	Tools / automation	Complete machine-readable dataset	Yes	Yes
HTML	Analyst / client	Interactive dashboard, all findings	Yes	Yes
PDF	Reporting / archive	Professional document, key tables	Yes	—

Table 8.1 — Report formats by run mode.

Reconnaissance Coverage by Module Category

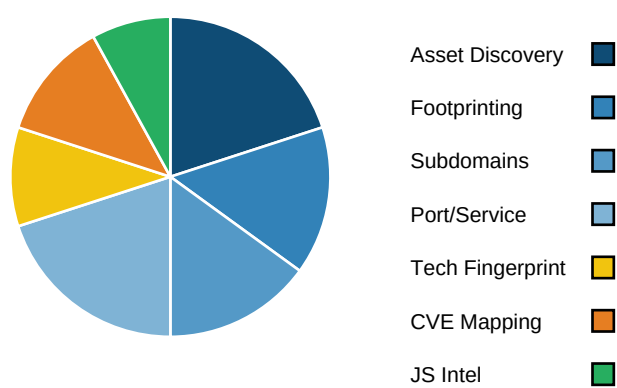


Figure 8.1 — Reconnaissance coverage contributed by each module category.

9. Limitations & Future Work

9.1 Known Limitations

- Hosting-provider detection relies on reverse DNS; providers without PTR records (e.g. Cloudflare) show as Unknown.
- OS detection is heuristic (TTL + open-port hints), not a substitute for deep stack fingerprinting.
- CVE correlation is keyword/version based and should be manually validated against the actual deployment.
- Public APIs (NVD, crt.sh, CIRCL) may rate-limit; results depend on source availability.

9.2 Planned Enhancements

- ASN / IP-WHOIS lookup for accurate hosting-provider attribution.
- Pluggable wordlists and additional passive sources for subdomain discovery.
- Optional API-key support to raise NVD rate limits.
- Configuration file / CLI flags for non-interactive automation.

9.3 Verification Status

All nine modules have been functionally tested against live targets, and the reporting pipeline has been validated across edge cases (empty, None-valued, IP-only, and untrusted/HTML-injection input). Three correctness bugs (CVE severity counting, empty TLS certificate fields, and local-vs-remote TTL reading) were identified and fixed during review.

10. About the Author

Author Profile	
Name	Md. Mostofa Nayon
Location	Dhaka, Bangladesh
Focus	Professional Master's in Information & Cyber Security, University of Dhaka
Education	B.Sc. in Computer Science & Engineering, Daffodil International University (2022–2026)
Certifications	AWS Academy – Cloud Foundations · Cisco CCNA · IT Infrastructure & Ethical Hacking (PGD) · AI+ Prompt Engineer L1
Core Skills	Python, C/C++, Java, JS/TS · REST APIs, Docker, Linux · AWS (EC2/S3/IAM) · TCP/IP, Routing, VLANs, MikroTik, Wireshark, Penetration Testing
Contact	contact@nayon.bd · +8801301994411
Web / Code	nayon.bd · github.com/MMNayon00

Table 10.1 — Author profile.

This framework was designed and developed by Md. Mostofa Nayon as part of applied study in information and cyber security, combining backend engineering with networking and offensive-security fundamentals. It reflects a defensive, authorization-first philosophy: comprehensive reconnaissance that informs remediation without ever crossing into exploitation.

Appendix A — Common Ports Reference

Port	Service	Port	Service	Port	Service
21	FTP	22	SSH	23	Telnet
25	SMTP	53	DNS	80	HTTP
110	POP3	135	MSRPC	139	NetBIOS
143	IMAP	443	HTTPS	445	SMB
993	IMAPS	995	POP3S	1433	MSSQL
3306	MySQL	3389	RDP	5432	PostgreSQL
8080	HTTP-Alt	8443	HTTPS-Alt	548	AFP
636	LDAPS	465	SMTPS	27017	MongoDB

Table A.1 — Representative ports referenced by the scanner and OS heuristics.

Appendix B — Glossary

Term	Definition
ASN	Autonomous System Number — identifies a network operator.
Certificate Transparency	Public logs of issued TLS certificates (source for crt.sh).
CVE	Common Vulnerabilities and Exposures — a catalogued vulnerability.
CVSS	Common Vulnerability Scoring System (0–10 severity scale).
Footprinting	Gathering ownership, encryption, and configuration details of a target.
PTR / Reverse DNS	Mapping an IP address back to a hostname.
SAN	Subject Alternative Name — extra hostnames covered by a certificate.
TTL	Time To Live — IP hop counter used for OS heuristics.

Table B.1 — Glossary of terms.